MANUAL TÉCNICO

Sistema de Gestión y Control de Laboratorios Universitarios – SIGELAB

El presente Manual Técnico detalla la arquitectura tecnológica, el modelo de datos, la lógica de negocio y las pautas de despliegue, mantenimiento y contingencia del Sistema de Gestión y Control de Laboratorios Universitarios – SIGELAB. Versión de documento: 1.0 — Emisión: octubre de 2026.

UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas

Índice

1. Propósito 3
2. Alcance 3
3. Arquitectura Técnica e Infraestructura 3
3.1. Especificaciones del Stack Tecnológico 3
3.2. Diagrama de Arquitectura de Despliegue 4
3.3. Organización del Código Fuente 5
4. Modelo de Datos y Diccionario 6
4.1. Diagrama Entidad-Relación (E-R) 6
4.2. Diccionario de Datos 10
5. Flujos de Procesos y Lógica de Negocio 15
5.1. Flujo Principal del Sistema 15
5.2. Ciclo de Vida del Equipo 17
5.3. Reglas de Negocio Implementadas 17
6. Guía de Despliegue, Mantenimiento y Contingencia 19
6.1. Requisitos Previos para el Despliegue 19
6.2. Procedimiento de Publicación / Despliegue 19
6.3. Plan de Mantenimiento y Respaldos (DRP / BCP) 21
7. Firmas y Control 22

1. Propósito

El presente Manual Técnico tiene como objetivo detallar la arquitectura tecnológica, el modelo de datos, la lógica de negocio y las pautas de instalación y administración del sistema informático denominado Sistema de Gestión y Control de Laboratorios Universitarios (SIGELAB), con la finalidad de facilitar su mantenimiento a largo plazo, apoyar la transferencia de conocimiento entre los integrantes del equipo de desarrollo y servir como referencia técnica durante la evaluación y defensa del Proyecto Final del curso 017 – Programación II.

2. Alcance

El documento está dirigido al catedrático del curso, a los integrantes del equipo de desarrollo y al personal técnico que deba instalar, configurar, mantener o ampliar la aplicación de escritorio SIGELAB. Describe la aplicación desarrollada en Java con interfaz Swing y su base de datos SigelabDB en Microsoft SQL Server, incluidos los módulos de inicio de sesión con validación facial, usuarios y roles, laboratorios, tipos de equipo, equipos con código QR, préstamos, devoluciones, mantenimiento, reportes y auditoría, así como la recuperación de contraseña por correo electrónico.

3. Arquitectura Técnica e Infraestructura

3.1. Especificaciones del Stack Tecnológico

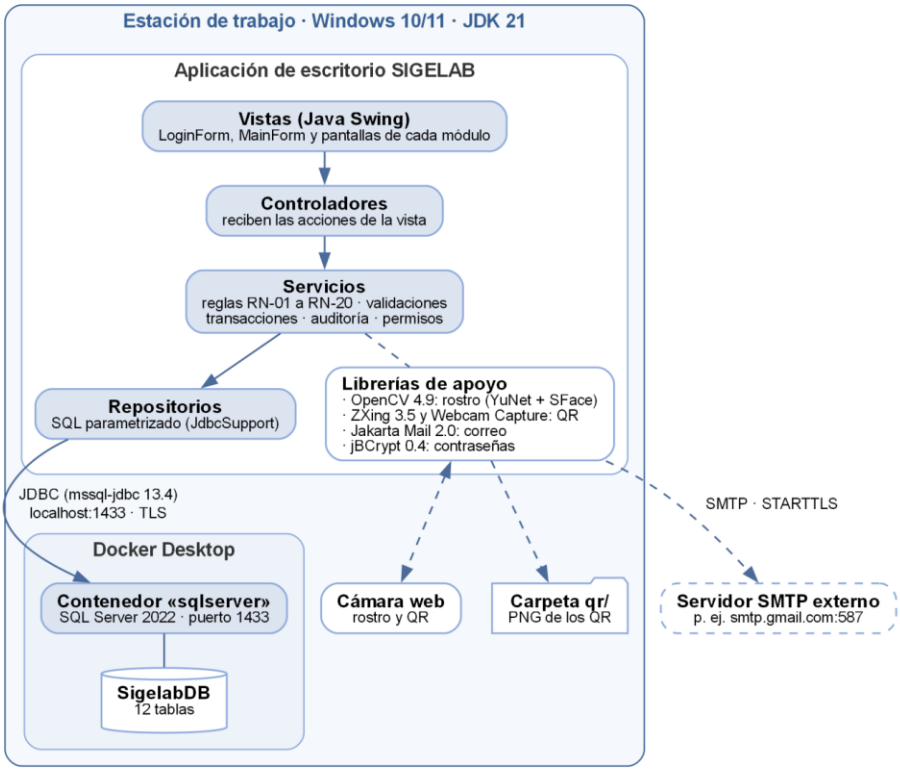
- **Lenguaje y plataforma:** Java 21 (LTS), compilado con release 21 sobre un JDK 21 (Eclipse Temurin u Oracle GraalVM).
- **Interfaz gráfica:** Java Swing, con formularios diseñados en el GUI Designer de IntelliJ IDEA (archivos .form, runtime forms_rt 7.0.3) y componentes visuales propios en tema oscuro (tarjetas, botones y campos con estilo).
- **Patrón de arquitectura:** arquitectura en capas basada en Modelo-Vista-Controlador: Vista → Controlador → Servicio → Repositorio → base de datos, con modelos (model) y enumeraciones (enums) compartidos por todas las capas.
- **Acceso a datos:** JDBC con Microsoft JDBC Driver for SQL Server 13.4.0 (mssql-jdbc 13.4.0.jre11), consultas parametrizadas centralizadas en JdbcSupport y transacciones explícitas con la clase Transaccion.
- **Motor de base de datos:** Microsoft SQL Server 2022 (imagen mcr.microsoft.com/mssql/server:2022-latest) en un contenedor de Docker.
- **Autenticación y seguridad:** contraseñas con hash BCrypt (jBCrypt 0.4); segundo factor por reconocimiento facial con OpenCV 4.9.0 (modelos YuNet para detección y SFace para reconocimiento); control de acceso por rol y por módulo.
- **Códigos QR:** ZXing 3.5.4 (core y javase) para generar y leer los códigos, y Webcam Capture 0.3.12 para leerlos con la cámara web.
- **Correo electrónico:** Jakarta Mail 2.0.1 sobre SMTP con STARTTLS, para enviar el código de recuperación de contraseña.
- **Gestión de dependencias:** Apache Maven (pom.xml) con maven-compiler-plugin 3.13.0.

- **Entorno de desarrollo:** IntelliJ IDEA, con la configuración de ejecución compartida SIGELAB (.run/SIGELAB.run.xml).
- **Control de versiones:** Git, con el repositorio remoto en GitHub (SaideVG/ProyectoLab_Prograll), rama principal main.
- **Ubicación del sistema informático:** local; la aplicación se ejecuta en la estación de trabajo y se conecta al servidor SQL Server del contenedor en localhost:1433.

3.2. Diagrama de Arquitectura de Despliegue

SIGELAB es una aplicación de escritorio de dos niveles: la aplicación Java se ejecuta en la estación de trabajo y se conecta directamente, por JDBC, a la base de datos SigelabDB alojada en Microsoft SQL Server, que corre dentro de un contenedor de Docker en el puerto 1433. No requiere servidor de aplicaciones ni servidor web.

Internamente la aplicación se organiza en capas: las vistas Swing reciben las acciones del usuario y las delegan a los controladores; estos invocan a los servicios, donde residen las reglas de negocio (RN-01 a RN-20), las validaciones y las transacciones; y los servicios utilizan los repositorios, únicos responsables de ejecutar SQL. La clase Sesion conserva al usuario conectado y valida el permiso de su rol antes de cada operación. Además, la aplicación usa la cámara web (validación facial y lectura de QR), el sistema de archivos (imágenes PNG de los códigos QR en la carpeta qr) y un servidor SMTP externo para los códigos de recuperación de contraseña.



Arquitectura de despliegue del sistema informático.

3.3. Organización del Código Fuente

El código fuente se encuentra en src/main/java, dentro del paquete universidad.sigelab, y suma 100 clases Java distribuidas por capa:

Paquete	Responsabilidad	Clases principales
universidad.sigelab	Punto de entrada: precarga OpenCV y abre el inicio de sesión.	Main
config	Conexión JDBC con la configuración externa (propiedad, variable de entorno o valor por defecto).	DatabaseConnection
view (29)	Pantallas Swing, diálogos y componentes visuales.	LoginForm, MainForm, EquipoView, PrestamoView, DevolucionView, MantenimientoView, ReporteView, AuditoriaView, UsuarioView, LaboratorioView, TipoEquipoView, LectorQrCamaraView, DialogoValidacionFacial, DialogoRegistroRostro, DialogoRecuperacion, RegistroUsuarioForm, Tema
controller (10)	Puente entre la vista y los servicios.	UsuarioController, LaboratorioController, TipoEquipoController, EquipoController, PrestamoController, DevolucionController, MantenimientoController, ReporteController, AuditoriaController, QrController
service (17)	Reglas de negocio, validaciones, transacciones y auditoría.	UsuarioService, PrestamoService, DevolucionService, MantenimientoService, EquipoService, ReporteService, AuditoriaService, ReconocimientoFacial, QrService, CamaraService, CorreoService, RecuperacionService
repository (11)	Único lugar con SQL; consultas parametrizadas.	UsuarioRepository, EquipoRepository, PrestamoRepository, DevolucionRepository, MantenimientoRepository, AuditoriaRepository, UsuarioRostroRepository, TokenRecuperacionRepository
model (14)	Entidades y objetos de consulta.	Usuario, Rol, Laboratorio, TipoEquipo, Equipo, Prestamo, DetallePrestamo, Devolucion, Mantenimiento, Auditoria, Dashboard, PrestamoResumen, HistorialEquipo, CodigoQr
enums (10)	Estados y catálogos fijos del sistema.	EstadoEquipo, EstadoPrestamo, EstadoMantenimiento, EstadoRegistro, CondicionDevolucion, TipoMantenimiento, ResultadoMantenimiento, RolUsuario, Modulo, AccionAuditoria
util (7)	Apoyo transversal.	Sesion, Transaccion, JdbcSupport, Validador, Fechas, BusinessException, DataAccessException
resources	Archivos de la aplicación.	images/logo_sigelab.png.png y rostro/ con los modelos ONNX de YuNet y SFace

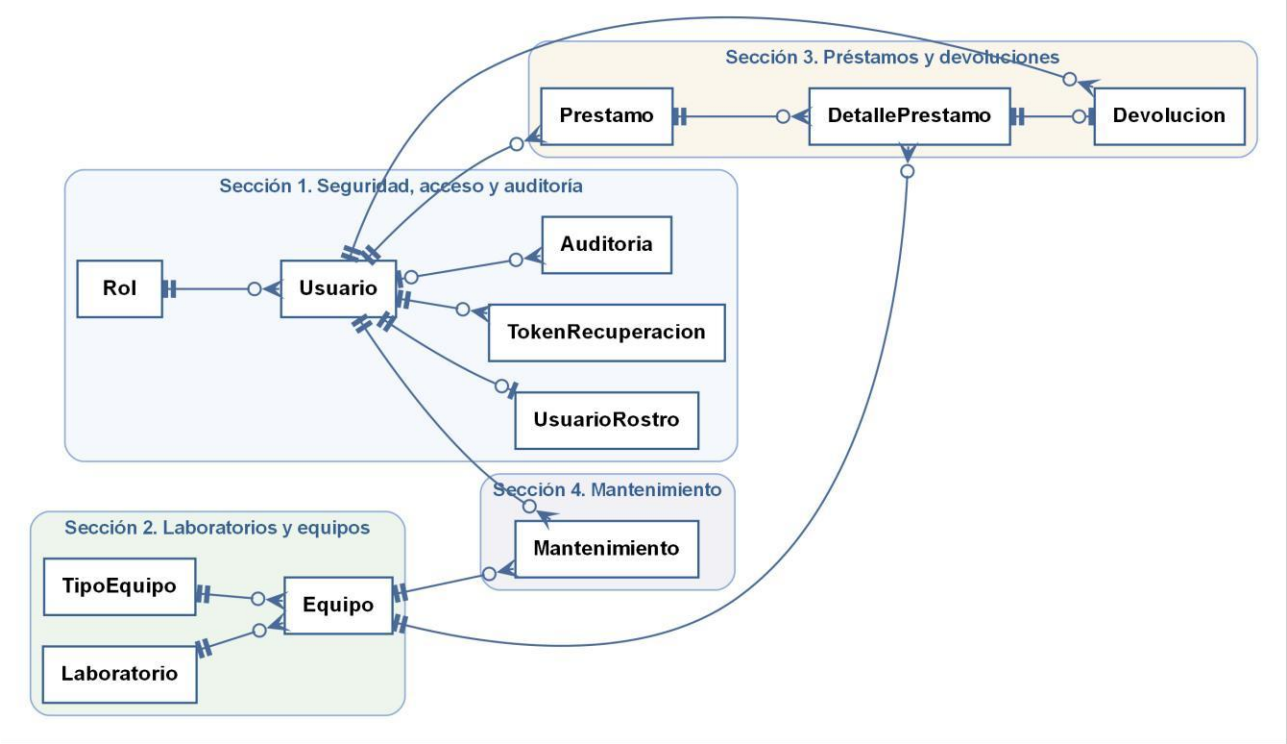
4. Modelo de Datos y Diccionario

4.1. Diagrama Entidad-Relación (E-R)

El modelo entidad-relación de SIGELAB está implementado en la base de datos SigelabDB de Microsoft SQL Server mediante el script database/01_esquema.sql; los datos de prueba se cargan con database/02_datos_prueba.sql. El modelo está conformado por doce tablas, con llaves primarias IDENTITY, llaves foráneas y restricciones UNIQUE, NOT NULL y CHECK que refuerzan en la base de datos las mismas reglas que aplica la capa de servicio.

Por su extensión, el diagrama se presenta en dos partes. Primero el mapa de referencia, que muestra todas las tablas agrupadas por sección y sus relaciones con notación de pata de gallo (una barra doble indica «uno y solo uno», el círculo indica «cero» y la pata de gallo indica «muchos»); luego las secciones ampliadas, con todos los campos de cada tabla, su tipo de dato y sus llaves (el asterisco marca los campos obligatorios).

Mapa de referencia del modelo

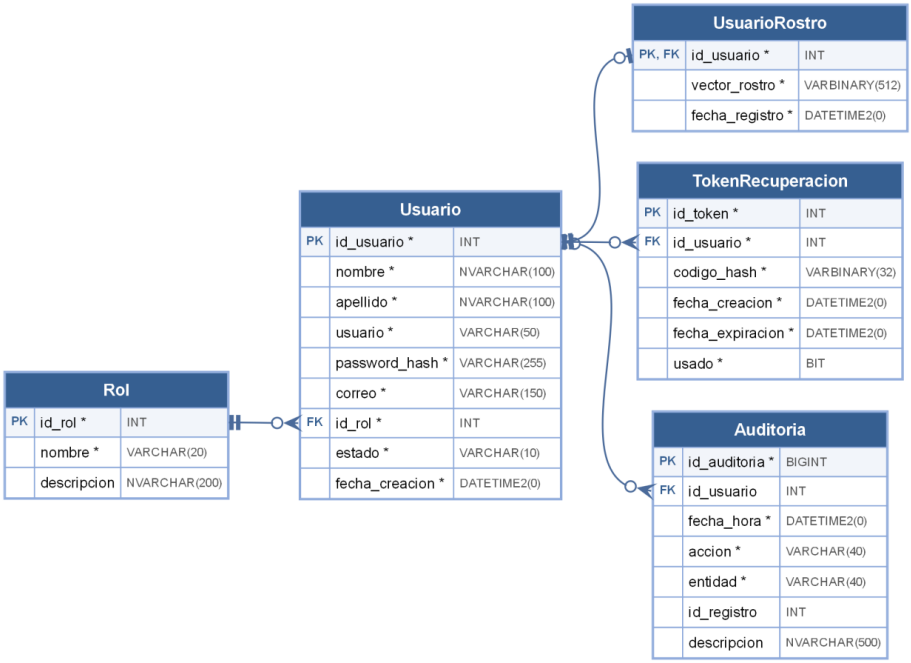


Mapa de referencia del modelo: tablas por sección y sus relaciones

Sección 1. Seguridad, acceso y auditoría

Agrupa las tablas que controlan quién puede entrar al sistema y qué hizo. Cada usuario tiene un rol, puede tener una firma facial para el segundo factor del inicio de sesión y códigos de recuperación de contraseña; la auditoría registra los eventos importantes asociados al usuario que los realizó.

Diagrama E-R — Sección 1. Seguridad, acceso y auditoría.
Tablas incluidas: ROL, USUARIO, USUARIOROSTRO, TOKENRECUPERACION, AUDITORIA.



Sección 2. Laboratorios y equipos

Contiene el inventario: cada equipo pertenece a un laboratorio y a un tipo de equipo (catálogo), tiene un código de inventario único que también es el contenido de su código QR y un estado que controla si puede prestarse.

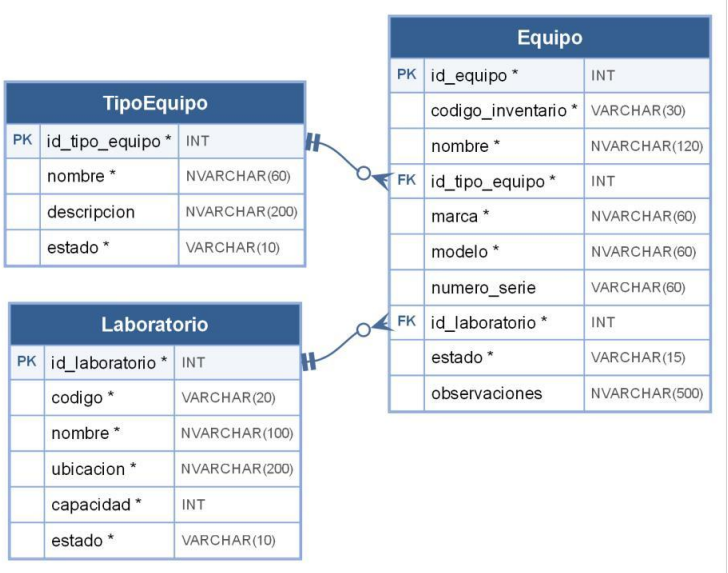


Diagrama E-R — Sección 2. Laboratorios y equipos.
Tablas incluidas: LABORATORIO, TIPOEQUIPO, EQUIPO.

Sección 3. Préstamos y devoluciones

El préstamo es un encabezado con uno o varios equipos (detalle). Cada equipo del detalle puede tener como máximo una devolución; un detalle sin devolución es un equipo pendiente. Las tablas en gris (Usuario y Equipo) se muestran solo con sus llaves, como referencia.

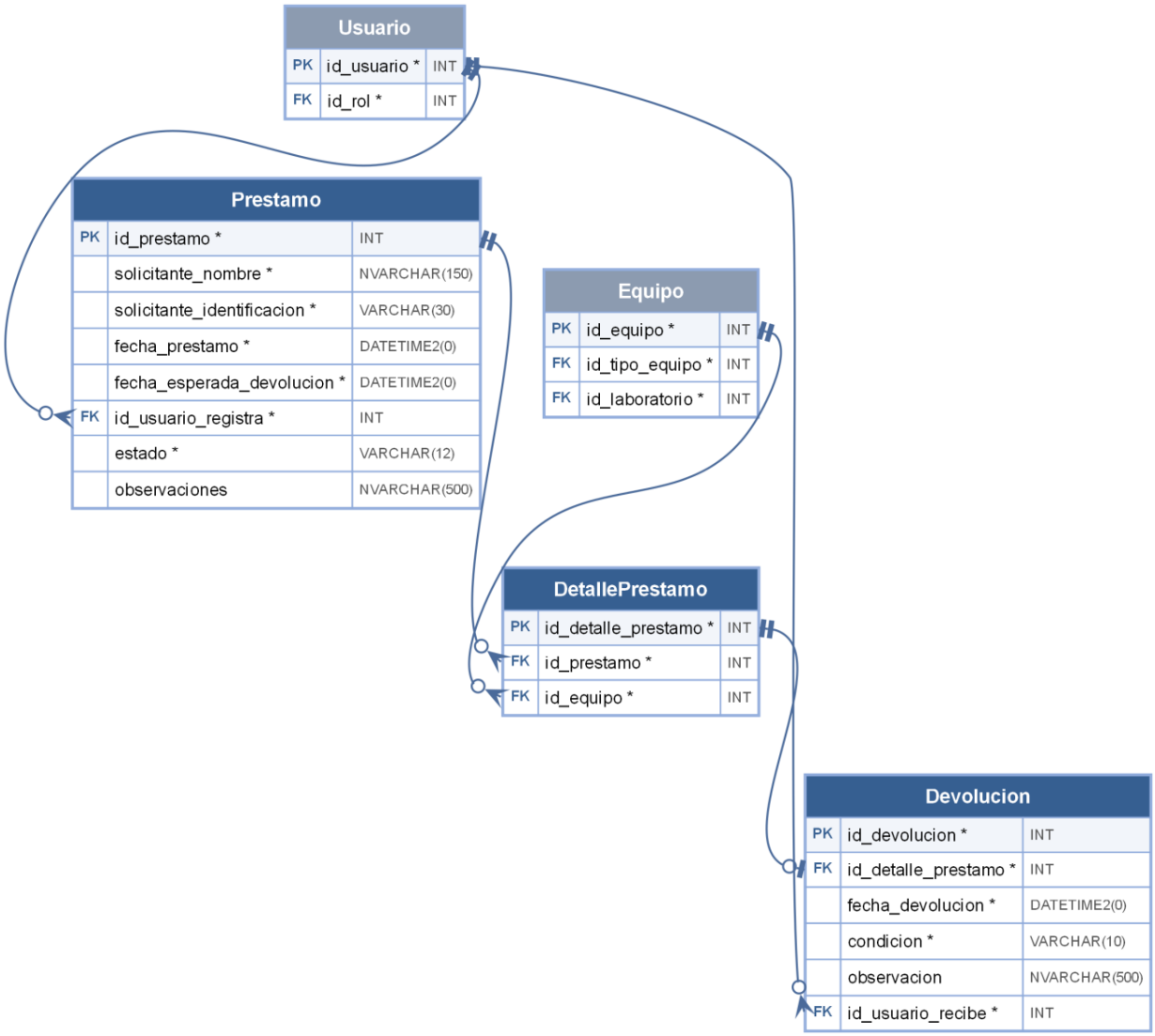


Diagrama E-R — Sección 3. Préstamos y devoluciones.
Tablas incluidas: PRESTAMO, DETALLEPRESTAMO, DEVOLUCION.

Sección 4. Mantenimiento

Historial de intervenciones de cada equipo, con el usuario que las registra. Un equipo puede tener varios mantenimientos a lo largo del tiempo, pero solo uno en proceso a la vez, porque mientras está en MANTENIMIENTO no puede volver a enviarse.

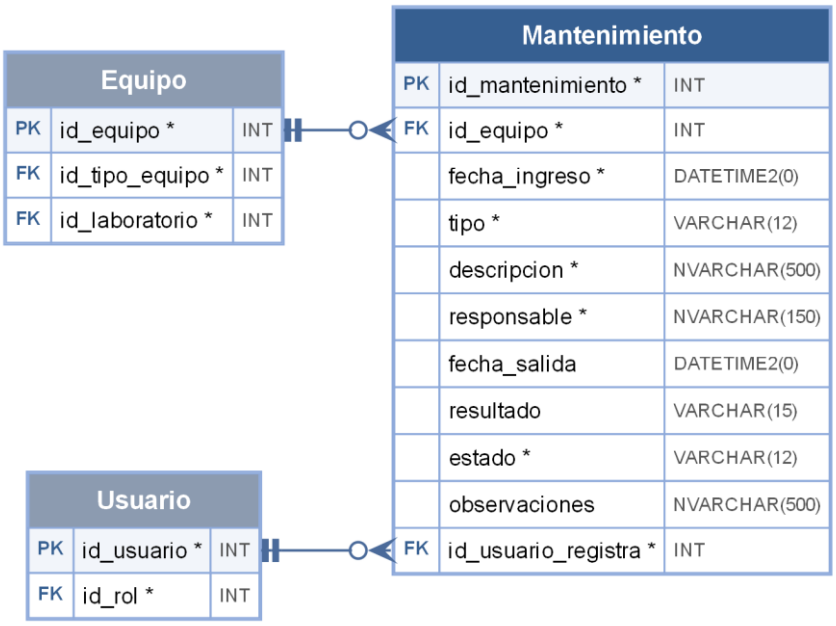


Diagrama E-R — Sección 4. Mantenimiento.
Tablas incluidas: MANTENIMIENTO.

4.2. Diccionario de Datos

El diccionario de datos describe cada una de las tablas de la base de datos SigelabDB, indicando la información que almacena, el detalle de sus campos y las restricciones definidas en el script. Las tablas se presentan agrupadas en las mismas secciones del diagrama entidad-relación.

En cada tabla de campos se utiliza la siguiente nomenclatura:

- **No.** Número correlativo del campo dentro de la tabla.
- **Campo.** Nombre del campo en la base de datos.
- **PK.** Indica que el campo forma parte de la llave primaria de la tabla.
- **FK.** Indica que el campo es llave foránea y hace referencia a otra tabla.
- **Obligatorio.** Indica si el campo requiere valor (NOT NULL).
- **Tipo de dato.** Tipo y longitud del campo en SQL Server; IDENTITY indica que el valor lo genera la base de datos de forma automática.
- **Restricciones.** Llaves únicas (UNIQUE), validaciones (CHECK), llaves foráneas y valores por defecto.

Sección 1. Seguridad, acceso y auditoría

Roles, usuarios con su firma facial y sus códigos de recuperación, y la bitácora de auditoría.

SIGELABDB.ROL

Descripción: Catálogo de los roles del sistema: ADMIN, ENCARGADO y DOCENTE. El rol determina los módulos a los que tiene acceso el usuario (enum RolUsuario).

No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_rol	Sí		Sí	INT IDENTITY
2	nombre			Sí	VARCHAR(20)
3	descripcion			No	NVARCHAR(200)

Restricciones: CHECK nombre IN ('ADMIN', 'ENCARGADO', 'DOCENTE'); UNIQUE (nombre).

SIGELABDB.USUARIO

Descripción: Usuarios que acceden al sistema, con sus datos personales, nombre de usuario, correo, rol y estado. La contraseña nunca se guarda en texto plano: password_hash contiene su hash BCrypt.

No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_usuario	Sí		Sí	INT IDENTITY
2	nombre			Sí	NVARCHAR(100)
3	apellido			Sí	NVARCHAR(100)
4	usuario			Sí	VARCHAR(50)
5	password_hash			Sí	VARCHAR(255)
6	correo			Sí	VARCHAR(150)
7	id_rol		Sí	Sí	INT
8	estado			Sí	VARCHAR(10)
9	fecha_creacion			Sí	DATETIME2(0)

Restricciones: CHECK estado IN ('ACTIVO', 'INACTIVO'); FK id_rol → Rol.id_rol; UNIQUE (correo); UNIQUE (usuario); Valores por defecto: estado = 'ACTIVO', fecha_creacion = SYSDATETIME().

SIGELABDB.USUARIOROSTRO					
Descripción: Firma facial del usuario para la verificación del segundo factor del inicio de sesión (RN-02): 128 valores float calculados por el modelo SFace (512 bytes). Relación uno a uno con Usuario.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_usuario	Sí	Sí	Sí	INT
2	vector_rostro			Sí	VARBINARY(512)
3	fecha_registro			Sí	DATETIME2(0)
Restricciones: FK id_usuario → Usuario.id_usuario; Valores por defecto: fecha_registro = SYSDATETIME().					

SIGELABDB.TOKENRECUPERACION					
Descripción: Códigos de un solo uso para restablecer la contraseña. Se guarda el hash SHA-256 del código de 6 dígitos, la fecha de vencimiento (10 minutos después de creado) y si ya fue utilizado.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_token	Sí		Sí	INT IDENTITY
2	id_usuario		Sí	Sí	INT
3	codigo_hash			Sí	VARBINARY(32)
4	fecha_creacion			Sí	DATETIME2(0)
5	fecha_expiracion			Sí	DATETIME2(0)
6	usado			Sí	BIT
Restricciones: FK id_usuario → Usuario.id_usuario; Valores por defecto: fecha_creacion = SYSDATETIME(), usado = 0.					

SIGELABDB.AUDITORIA					
Descripción: Bitácora de trazabilidad (RN-18): quién, cuándo y qué operación se realizó (inicios de sesión, préstamos, devoluciones, mantenimientos, cambios de usuario, de rostro y recuperación de contraseña). id_usuario admite NULL para los intentos de inicio de sesión con usuarios que no existen.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_auditoria	Sí		Sí	BIGINT IDENTITY
2	id_usuario		Sí	No	INT
3	fecha_hora			Sí	DATETIME2(0)
4	accion			Sí	VARCHAR(40)
5	entidad			Sí	VARCHAR(40)
6	id_registro			No	INT
7	descripcion			No	NVARCHAR(500)
Restricciones: FK id_usuario → Usuario.id_usuario; Valores por defecto: fecha_hora = SYSDATETIME().					

Sección 2. Laboratorios y equipos

Ubicación de los equipos, catálogo de tipos y ficha de cada equipo con su estado.

SIGELABDB.LABORATORIO					
Descripción: Ubicación lógica de los equipos. El código es único (RN-03) y la capacidad debe ser mayor que cero. Los laboratorios se activan o desactivan; no se eliminan.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_laboratorio	Sí		Sí	INT IDENTITY
2	codigo			Sí	VARCHAR(20)
3	nombre			Sí	NVARCHAR(100)
4	ubicacion			Sí	NVARCHAR(200)
5	capacidad			Sí	INT
6	estado			Sí	VARCHAR(10)
Restricciones: CHECK capacidad>0; CHECK estado IN ('ACTIVO', 'INACTIVO'); UNIQUE (codigo); Valores por defecto: estado = 'ACTIVO'.					

SIGELABDB.TIPOEQUIPO					
Descripción: Catálogo de categorías de equipo (Laptop, Cámara, Proyector, Router, Switch, Tablet, Monitor, Kit electrónico). El nombre es único.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_tipo_equipo	Sí		Sí	INT IDENTITY
2	nombre			Sí	NVARCHAR(60)
3	descripcion			No	NVARCHAR(200)
4	estado			Sí	VARCHAR(10)
Restricciones: CHECK estado IN ('ACTIVO', 'INACTIVO'); UNIQUE (nombre); Valores por defecto: estado = 'ACTIVO'.					

SIGELABDB.EQUIPO					
Descripción: Activo principal administrado. El código de inventario es único (RN-04) y es el contenido de su código QR. El estado sigue el ciclo DISPONIBLE, PRESTADO, DANADO, MANTENIMIENTO y BAJA; se escribe DANADO, sin ñ, igual que el enum EstadoEquipo.					
No.	**Campo**	**PK**	**FK**	**Obligatorio**	**Tipo de dato**
1	id_equipo	Sí		Sí	INT IDENTITY
2	codigo_inventario			Sí	VARCHAR(30)
3	nombre			Sí	NVARCHAR(120)
4	id_tipo_equipo		Sí	Sí	INT
5	marca			Sí	NVARCHAR(60)
6	modelo			Sí	NVARCHAR(60)
7	numero_serie			No	VARCHAR(60)
8	id_laboratorio		Sí	Sí	INT
9	estado			Sí	VARCHAR(15)
10	observaciones			No	NVARCHAR(500)
Restricciones: CHECK estado IN ('DISPONIBLE', 'PRESTADO', 'DANADO', 'MANTENIMIENTO', 'BAJA'); FK id_laboratorio → Laboratorio.id_laboratorio; FK id_tipo_equipo → TipoEquipo.id_tipo_equipo; UNIQUE (codigo_inventario); Valores por defecto: estado = 'DISPONIBLE'.					

Sección 3. Préstamos y devoluciones

Encabezado del préstamo, equipos incluidos y devolución de cada uno.

SIGELABDB.PRESTAMO

Descripción: Encabezado del préstamo: solicitante, fechas y usuario que lo registra. Permanece ACTIVO mientras tenga equipos pendientes (RN-16) y pasa a FINALIZADO con la última devolución. El estado vencido no se guarda: se calcula al consultar (RN-17).

No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_prestamo	Sí		Sí	INT IDENTITY
2	solicitante_nombre			Sí	NVARCHAR(150)
3	solicitante_identificacion			Sí	VARCHAR(30)
4	fecha_prestamo			Sí	DATETIME2(0)
5	fecha_esperada_devolucion			Sí	DATETIME2(0)
6	id_usuario_registra		Sí	Sí	INT
7	estado			Sí	VARCHAR(12)
8	observaciones			No	NVARCHAR(500)

Restricciones: CHECK estado IN ('ACTIVO', 'FINALIZADO'); CHECK fecha_esperada_devolucion>=fecha_prestamo; FK id_usuario_registra → Usuario.id_usuario; Valores por defecto: fecha_prestamo = SYSDATETIME(), estado = 'ACTIVO'.

SIGELABDB.DETALLEPRESTAMO

Descripción: Resuelve la relación de muchos a muchos entre Prestamo y Equipo. La llave única (id_prestamo, id_equipo) impide repetir un equipo en el mismo préstamo (RN-06). Un detalle sin devolución es un equipo pendiente.

No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_detalle_prestamo	Sí		Sí	INT IDENTITY
2	id_prestamo		Sí	Sí	INT
3	id_equipo		Sí	Sí	INT

Restricciones: FK id_equipo → Equipo.id_equipo; FK id_prestamo → Prestamo.id_prestamo; UNIQUE (id_prestamo, id_equipo).

SIGELABDB.DEVOLUCION

Descripción: Una fila por cada equipo devuelto, con la fecha, la condición (BUENO o DANADO) y el usuario que lo recibe. La llave única sobre id_detalle_prestamo impide devolver dos veces el mismo equipo (RN-11).

No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_devolucion	Sí		Sí	INT IDENTITY
2	id_detalle_prestamo		Sí	Sí	INT
3	fecha_devolucion			Sí	DATETIME2(0)
4	condicion			Sí	VARCHAR(10)
5	observacion			No	NVARCHAR(500)
6	id_usuario_recibe		Sí	Sí	INT

Restricciones: CHECK condicion IN ('BUENO', 'DANADO'); FK id_detalle_prestamo → DetallePrestamo.id_detalle_prestamo; FK id_usuario_recibe → Usuario.id_usuario; UNIQUE (id_detalle_prestamo); Valores por defecto: fecha_devolucion = SYSDATETIME().

Sección 4. Mantenimiento

Historial de intervenciones preventivas y correctivas de cada equipo.

SIGELABDB.MANTENIMIENTO

Descripción: Intervenciones de un equipo: tipo (PREVENTIVO o CORRECTIVO), descripción, responsable, fechas de ingreso y salida, estado (EN_PROCESO o FINALIZADO) y resultado (REPARADO o NO_REPARABLE). Un CHECK exige que un mantenimiento finalizado tenga fecha de salida y resultado, y que uno en proceso no los tenga.

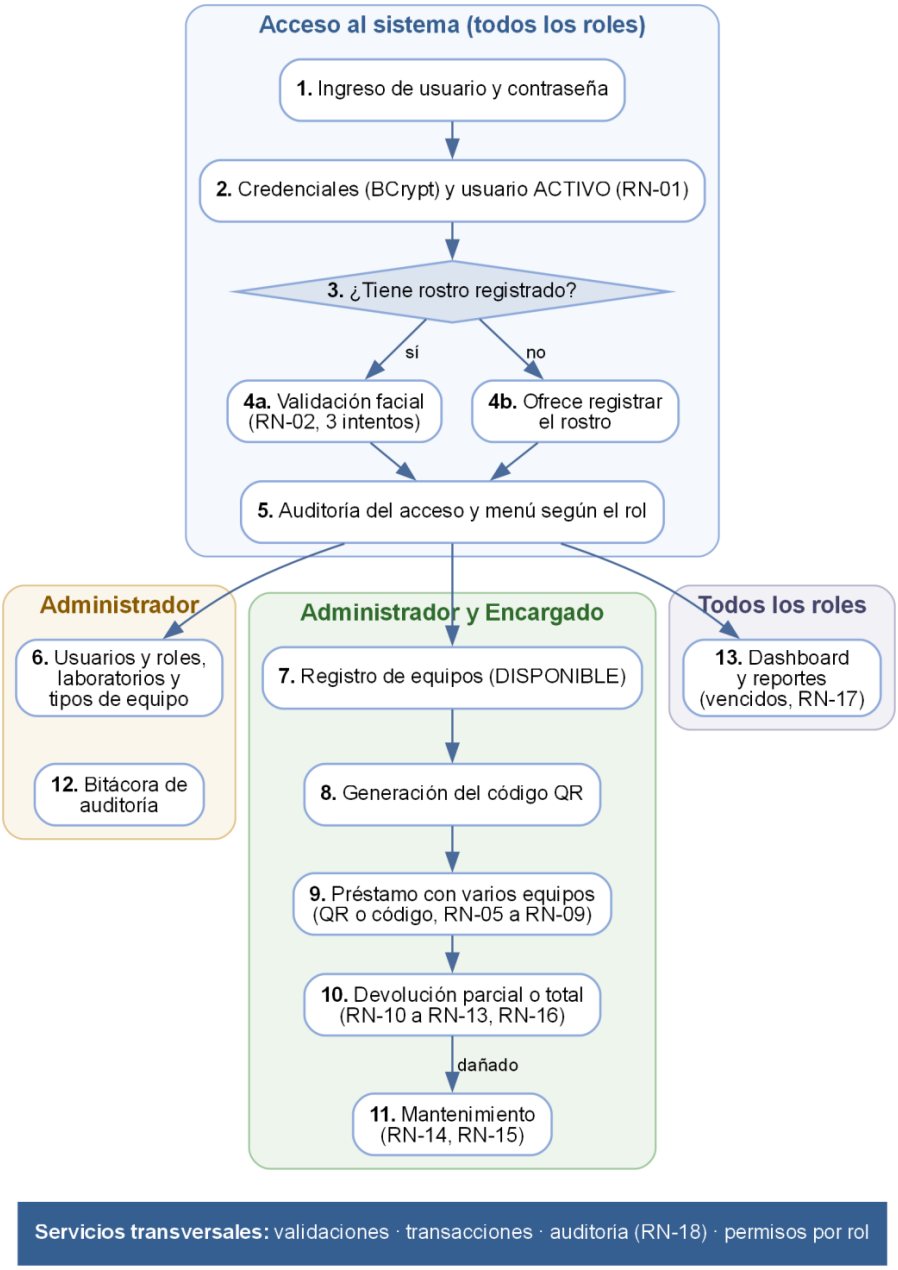
No.	Campo	PK	FK	Obligatorio	Tipo de dato
1	id_mantenimiento	Sí		Sí	INT IDENTITY
2	id_equipo		Sí	Sí	INT
3	fecha_ingreso			Sí	DATETIME2(0)
4	tipo			Sí	VARCHAR(12)
5	descripcion			Sí	NVARCHAR(500)
6	responsable			Sí	NVARCHAR(150)
7	fecha_salida			No	DATETIME2(0)
8	resultado			No	VARCHAR(15)
9	estado			Sí	VARCHAR(12)
10	observaciones			No	NVARCHAR(500)
11	id_usuario_registra		Sí	Sí	INT

Restricciones: CHECK estado='EN_PROCESO' AND fecha_salida IS NULL AND resultado IS NULL OR estado='FINALIZADO' AND fecha_salida IS NOT NULL AND resultado IS NOT NULL; CHECK estado IN ('EN_PROCESO', 'FINALIZADO'); CHECK fecha_salida IS NULL OR fecha_salida>=fecha_ingreso; CHECK resultado IS NULL OR (resultado='NO_REPARABLE' OR resultado='REPARADO'); CHECK tipo IN ('PREVENTIVO', 'CORRECTIVO'); FK id_equipo → Equipo.id_equipo; FK id_usuario_registra → Usuario.id_usuario; Valores por defecto: fecha_ingreso = SYSDATETIME(), estado = 'EN_PROCESO'.

5. Flujos de Procesos y Lógica de Negocio

5.1. Flujo Principal del Sistema

SIGELAB controla los equipos de los laboratorios desde que se registran hasta que se dan de baja. El siguiente esquema resume el recorrido completo de un usuario y la participación de cada rol; el número de cada paso se explica en la descripción de las etapas.



Flujo general del sistema informático con la participación de cada rol.

Descripción de las etapas

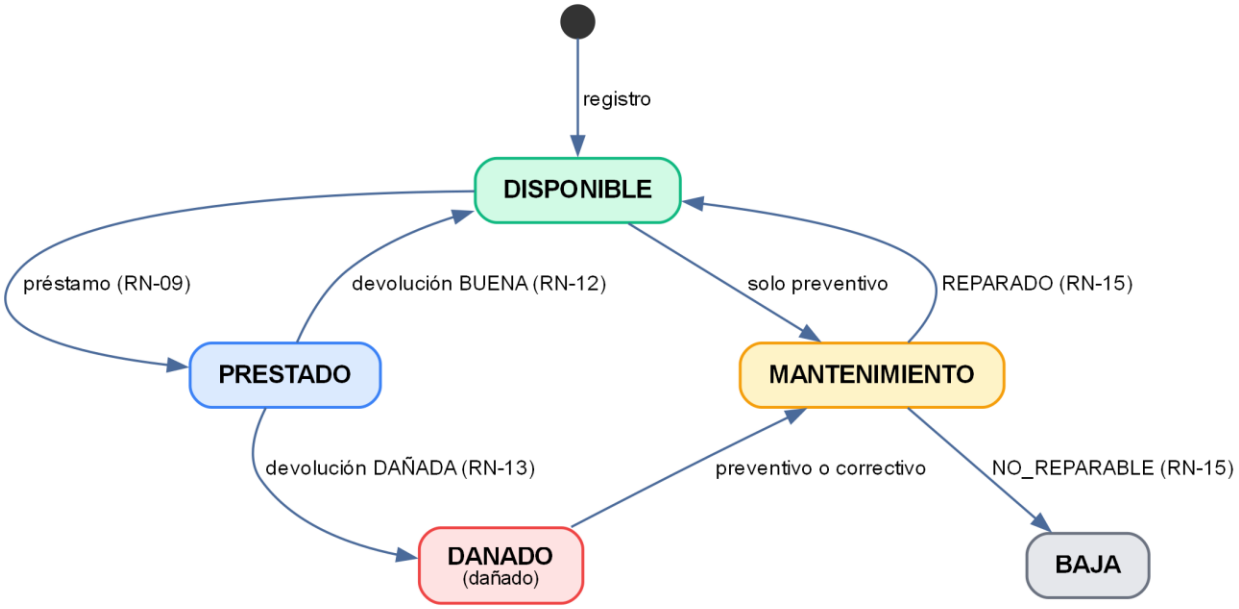
1. **Inicio de sesión (pasos 1 y 2).** El usuario escribe su usuario y contraseña en LoginForm. UsuarioService busca al usuario, compara la contraseña con su hash BCrypt y verifica que esté ACTIVO (RN-01). Cada intento fallido queda registrado en Auditoría con su motivo.
2. **Validación facial (pasos 3 y 4).** Si el usuario tiene una firma facial registrada, el inicio de sesión exige el segundo factor (RN-02): la cámara captura el rostro, YuNet lo detecta, SFace calcula la firma y se compara con la guardada mediante similitud del coseno (umbral 0,363), con hasta tres intentos. Si todavía no tiene rostro, el sistema le ofrece registrarlo.
3. **Menú principal (paso 5).** Se registra el acceso exitoso y MainForm muestra el dashboard y solo las tarjetas de los módulos permitidos para el rol. Además, cada servicio vuelve a validar el permiso con Sesion.exigirAcceso antes de ejecutar la operación.
4. **Catálogos (paso 6).** El administrador mantiene los usuarios y sus roles, los laboratorios (código único, RN-03) y los tipos de equipo. Estos registros se activan o desactivan; no se eliminan. El identificador (ID) de cada registro lo asigna la base de datos al guardar, por lo que no se muestra en los formularios de captura, pero sí en las tablas de resultados.
5. **Equipos y código QR (pasos 7 y 8).** Cada equipo se registra con un código de inventario único (RN-04), su tipo y su laboratorio, y comienza en estado DISPONIBLE. Desde la pantalla de equipos se genera su código QR (ZXing, 300 × 300 píxeles), que contiene el código de inventario y se guarda como imagen PNG en la carpeta qr.
6. **Préstamo (paso 9).** El encargado agrega los equipos escribiendo el código o leyendo el QR desde una imagen o con la cámara; la lectura solo identifica el equipo (RN-19). Solo se aceptan equipos DISPONIBLES y sin repetir (RN-06, RN-07, RN-08 y RN-14) y el préstamo debe tener al menos uno (RN-05). Al confirmar, PrestamoService guarda el encabezado y los detalles, cambia cada equipo a PRESTADO (RN-09) y registra la auditoría en una sola transacción.
7. **Devolución (paso 10).** Se elige un préstamo ACTIVO y se devuelve cada equipo pendiente (RN-10 y RN-11) indicando su condición: BUENO lo regresa a DISPONIBLE (RN-12) y DAÑADO lo deja en DANADO (RN-13). El préstamo sigue ACTIVO mientras quede algún equipo pendiente y pasa a FINALIZADO con la última devolución (RN-16).
8. **Mantenimiento (paso 11).** Un equipo DANADO se envía a mantenimiento preventivo o correctivo, y uno DISPONIBLE solo a preventivo; mientras está en MANTENIMIENTO no puede prestarse (RN-14). Al finalizar, el resultado REPARADO lo devuelve a DISPONIBLE y NO_REPARABLE lo da de BAJA (RN-15).
9. **Auditoría, dashboard y reportes (pasos 12 y 13).** El administrador consulta la bitácora filtrando por acción. Todos los roles ven el dashboard (equipos por estado, préstamos activos y vencidos) y los reportes: equipos por estado; préstamos activos, finalizados, vencidos (RN-17) y entre fechas; e historial de préstamos y de mantenimiento de un equipo.
10. **Servicios transversales (banda inferior).** Toda operación se valida en la capa de servicio antes de llegar a la base de datos (RN-20). Las operaciones compuestas se ejecutan en una transacción que se confirma o se revierte completa (Transaccion) y dejan su registro de Auditoría dentro de la misma transacción (RN-18). Los errores técnicos se traducen a mensajes comprensibles mediante BusinessException y DataAccessException.

Roles y responsabilidades

Rol	Responsabilidad en el flujo
Administrador (ADMIN)	Acceso a todos los módulos: usuarios y roles, laboratorios, tipos de equipo, equipos y QR, préstamos, devoluciones, mantenimiento, reportes y auditoría.
Encargado (ENCARGADO)	Operación diaria del laboratorio: equipos y QR, préstamos, devoluciones, mantenimiento y reportes.
Docente (DOCENTE)	Consulta del dashboard y de los reportes. Es el rol que recibe automáticamente quien crea su cuenta desde la opción «Crear cuenta» del inicio de sesión.

5.2. Ciclo de Vida del Equipo

El estado del equipo es el centro de las reglas de negocio: determina si puede prestarse, devolverse o enviarse a mantenimiento. El siguiente diagrama muestra los cambios de estado permitidos y la regla que los provoca.



Estados del equipo y reglas de negocio que provocan cada cambio.

Cada cambio de estado se hace con una actualización condicional (EquipoRepository.cambiarEstado: UPDATE … WHERE estado = estado esperado) dentro de la misma transacción de la operación. Si en ese instante otro usuario ya cambió el equipo, la actualización no afecta ninguna fila y la operación completa se rechaza, en lugar de dejar datos inconsistentes.

5.3. Reglas de Negocio Implementadas

Las veinte reglas de negocio obligatorias del proyecto se aplican en la capa de servicio (RN-20) y, cuando es posible, también en la base de datos:

Código	Regla	Dónde se aplica
RN-01	No se puede iniciar sesión con un usuario INACTIVO.	UsuarioService.verificarCredenciales

Código	Regla	Dónde se aplica
RN-02	La validación facial debe aprobarse para completar el inicio de sesión.	UsuarioService.autenticar, ReconocimientoFacial, DialogoValidacionFacial
RN-03	El código de laboratorio debe ser único.	LaboratorioService; UNIQUE UQ_Laboratorio_Codigo
RN-04	El código de inventario del equipo debe ser único.	EquipoService; UNIQUE UQ_Equipo_CodigoInventario
RN-05	Un préstamo debe tener al menos un equipo.	PrestamoService.registrarPrestamo
RN-06	El mismo equipo no puede aparecer dos veces en el mismo préstamo.	PrestamoService; UNIQUE UQ_DetallePrestamo
RN-07	Solo un equipo DISPONIBLE puede agregarse a un préstamo.	PrestamoService.validarEquipoParaPrestamo
RN-08	PRESTADO, DAÑADO, MANTENIMIENTO y BAJA bloquean nuevos préstamos.	PrestamoService.validarEquipoParaPrestamo
RN-09	Al confirmar un préstamo, cada equipo cambia automáticamente a PRESTADO.	PrestamoService.registrarPrestamo (transacción)
RN-10	Un equipo solo puede devolverse si está pendiente dentro de un préstamo activo.	DevolucionService.registrarDevolucion
RN-11	Un equipo no puede devolverse dos veces.	DevolucionService; UNIQUE UQ_Devolucion_Detalle
RN-12	Devolución BUENA cambia PRESTADO → DISPONIBLE.	DevolucionService.registrarDevolucion
RN-13	Devolución DAÑADA cambia PRESTADO → DAÑADO.	DevolucionService.registrarDevolucion
RN-14	Un equipo en MANTENIMIENTO no puede prestarse.	PrestamoService.validarEquipoParaPrestamo
RN-15	Al finalizar el mantenimiento, el equipo queda DISPONIBLE o de BAJA según el resultado.	MantenimientoService.finalizar
RN-16	Un préstamo con devoluciones parciales permanece activo mientras exista un equipo pendiente.	DevolucionService.registrarDevolucion
RN-17	Un préstamo vencido tiene la fecha esperada vencida y equipos pendientes.	PrestamoRepository.listarVencidos, PrestamoResumen
RN-18	Las operaciones relevantes deben dejar rastro en Auditoría.	AuditoriaService, dentro de cada transacción
RN-19	La lectura QR identifica un equipo; no modifica estados por sí sola.	QrService.leerCodigo y buscarCodigo
RN-20	Las reglas se ejecutan en la lógica de negocio y no dependen solo de la interfaz.	Capa service y restricciones de SigelabDB

6. Guía de Despliegue, Mantenimiento y Contingencia

6.1. Requisitos Previos para el Despliegue

- Estación de trabajo con Windows 10 u 11 de 64 bits, 8 GB de RAM como mínimo (16 GB recomendados) y cámara web para la validación facial y la lectura de QR con cámara.
- JDK 21 (Eclipse Temurin 21 u Oracle GraalVM 21) e IntelliJ IDEA con su soporte de Maven.
- Docker Desktop con un contenedor de Microsoft SQL Server 2022 (imagen mcr.microsoft.com/mssql/server:2022-latest) publicado en el puerto 1433.
- Git para obtener el código fuente desde el repositorio de GitHub.
- Base de datos SigelabDB creada con los scripts de la carpeta database del proyecto.
- Conexión a internet la primera vez, para que Maven descargue las dependencias (OpenCV ocupa unos 100 MB).
- Opcional: una cuenta de correo con SMTP (por ejemplo, una contraseña de aplicación de Gmail) para habilitar la recuperación de contraseña por correo.

6.2. Procedimiento de Publicación / Despliegue

SIGELAB se ejecuta desde su código fuente: no se publica en un servidor web ni requiere un servidor de aplicaciones. El despliegue consiste en obtener el código del repositorio Git, preparar la base de datos SigelabDB en SQL Server, configurar la conexión y ejecutar la aplicación desde IntelliJ IDEA.

Bajo este esquema, los datos que cambian entre computadoras —contraseña de la base de datos, servidor y cuenta de correo— no se escriben en el código: se leen de la configuración externa, en este orden de prioridad: propiedad de Java (-D), variable de entorno y, por último, el valor por defecto.

Variable	Uso	Valor por defecto
DB_URL	Cadena de conexión JDBC de la base de datos.	jdbc:sqlserver://localhost:1433; databaseName=SigelabDB; encrypt=true; trustServerCertificate=true; sendTimeAsDatetime=false;
DB_USER	Usuario de SQL Server.	sa
DB_PASSWORD	Contraseña de SQL Server.	ProyecLAB26
SIGELAB_SMTP_HOST	Servidor SMTP; sin este valor el envío de correo queda deshabilitado.	(sin valor)
SIGELAB_SMTP_PORT	Puerto del servidor SMTP.	587
SIGELAB_SMTP_USER	Cuenta del correo remitente.	(sin valor)
SIGELAB_SMTP_PASSWORD	Contraseña de aplicación de esa cuenta.	(sin valor)
SIGELAB_SMTP_FROM	Dirección que aparece como remitente.	la de SIGELAB_SMTP_USER
SIGELAB_SMTP_STARTTLS	Cifrado STARTTLS (con el puerto 587).	true

Variable	Uso	Valor por defecto
SIGELAB_SMTP_SSL	Cifrado SSL (con el puerto 465).	false

El repositorio se organiza con las siguientes ramas y convenciones:

- **Rama main:** contiene la versión integrada y estable del proyecto; desde ella se clona y se ejecuta el sistema.
- **Ramas de trabajo:** cada integrante desarrolla en su propia rama (por ejemplo, Login, rama-vistas o feature/rediseno-vistas) y la integra a main cuando su cambio funciona.
- **Mensajes de commit:** se sigue la convención del curso: feat: para funcionalidades nuevas y fix: para correcciones (por ejemplo, «feat: integrar lectura QR»).

Procedimiento de despliegue

1. Clonar el repositorio. Este paso se realiza únicamente la primera vez:
```
git clone https://github.com/SaideVG/ProyectoLab_Prograll.git
```
2. Crear el contenedor de SQL Server (solo la primera vez), con la contraseña que usa la aplicación por defecto:
```
docker run -d --name sqlserver -p 1433:1433 -e "ACCEPT_EULA=Y" -e
"MSSQL_SA_PASSWORD=ProyecLAB26" mcr.microsoft.com/mssql/server:2022-latest
```
3. Crear la base de datos SigelabDB con su esquema y los datos de prueba, desde la carpeta del proyecto:
```
.\database\cargar_en_docker.ps1
```
4. Si el contenedor usa otra contraseña o se habilitará el correo, definir las variables de entorno y abrir una terminal nueva:
```
setx DB_PASSWORD "<contraseña>"
setx SIGELAB_SMTP_HOST "smtp.gmail.com"
```
5. Abrir la carpeta del proyecto en IntelliJ IDEA, verificar en File > Project Structure que el SDK del proyecto sea un JDK 21 y ejecutar Reload All Maven Projects en el panel Maven.
6. Compilar con Build > Rebuild Project. Se recomienda compilar desde IntelliJ IDEA, porque su compilador instrumenta los formularios .form del GUI Designer de los que dependen algunas pantallas (por ejemplo, el lector de QR con cámara).
7. Ejecutar la configuración compartida SIGELAB, que inicia la clase universidad.sigelab.Main con la opción de la máquina virtual que requiere OpenCV:
```
--enable-native-access=ALL-UNNAMED
```
8. Iniciar sesión con un usuario de prueba (admin / Admin2026!) y registrar el rostro cuando el sistema lo ofrezca.
9. Para publicar cambios en el repositorio:
```
git pull
git add .
git commit -m "feat: descripción del cambio"
git push origin main
```

Importante: antes de la entrega final debe eliminarse de UsuarioService el usuario de prueba admin / admin (constantes USUARIO_PRUEBA y PASSWORD_PRUEBA), que permite entrar sin validar la contraseña guardada en la base de datos.

6.3. Plan de Mantenimiento y Respaldos (DRP / BCP)

- **Frecuencia de Respaldos:** respaldo completo de SigelabDB al final de cada jornada de préstamos y antes de cualquier cambio en el esquema o actualización del sistema.
- **Procedimiento de Respaldo:** se ejecuta BACKUP DATABASE dentro del contenedor y el archivo se copia fuera de él:

```
docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -P "<contraseña>"
-Q
"BACKUP DATABASE SigelabDB TO DISK='/var/opt/mssql/data/SigelabDB.bak' WITH INIT"
docker cp sqlserver:/var/opt/mssql/data/SigelabDB.bak C:\Respaldos\SigelabDB_AAAAMMDD.bak
```

- **Ruta de Almacenamiento:** los archivos .bak se guardan fuera del contenedor (por ejemplo, C:\Respaldos) y una copia en el almacenamiento en la nube del equipo; se conservan al menos los últimos siete respaldos. Si el contenedor se creó sin un volumen de datos, al eliminarlo con docker rm se pierde la base; por eso se recomienda crearlo con un volumen (-v sigelab_datos:/var/opt/mssql) además de mantener los respaldos.
- **Procedimiento de Restauración en Contingencia:** con la aplicación cerrada, se copia el respaldo al contenedor y se restaura sobre la base existente:

```
docker cp C:\Respaldos\SigelabDB_AAAAMMDD.bak sqlserver:/var/opt/mssql/data/SigelabDB.bak
docker exec sqlserver /opt/mssql-tools18/bin/sqlcmd -S localhost -U sa -C -P "<contraseña>"
-Q
"RESTORE DATABASE SigelabDB FROM DISK='/var/opt/mssql/data/SigelabDB.bak' WITH REPLACE"
```

- **Sin respaldo disponible:** la base puede recrearse con los scripts de la carpeta database (paso 3 del procedimiento de despliegue); se conservan los catálogos de prueba, pero se pierden los movimientos registrados.
- **Contingencias de la Aplicación:** si no hay cámara, el código QR puede leerse desde la imagen PNG. Si el reconocimiento facial no está disponible, un usuario con rostro registrado no puede completar el inicio de sesión (el sistema falla de forma segura); para habilitarlo temporalmente, el administrador de la base de datos elimina su firma facial y el usuario la registra de nuevo al ingresar:

```
DELETE FROM UsuarioRostro WHERE id_usuario = <id>;
```

- **Correo y Base de Datos:** sin servidor SMTP configurado, la recuperación de contraseña por correo no está disponible y el administrador restablece la contraseña desde el módulo Usuarios. Si la base de datos está detenida, la aplicación muestra el indicador de conexión en rojo y mensajes de error controlados; se inicia con:

```
docker start sqlserver
```

- **Mantenimiento Periódico:** revisar en Auditoría los inicios de sesión fallidos, mantener actualizados el JDK 21 y las dependencias del pom.xml y depurar los códigos de recuperación usados o vencidos:

```
DELETE FROM TokenRecuperacion WHERE usado = 1 OR fecha_expiracion < SYSDATETIME();
```

7. Firmas y Control

El presente manual técnico ha sido revisado y validado en sus componentes técnicos y operativos, apoyando la transferencia de conocimiento tecnológico y la preservación de los activos de software del Proyecto Final del curso 017 – Programación II de la Universidad Mariano Gálvez de Guatemala.